

Sistema de Inhabilitación de Clientes Cruzimex Ltda.

Documentación Técnica v2.0

Ludwing Julian Herrera Justiniano
Ing. de Sistemas - Desarrollador Full Stack Web

25 de noviembre de 2025

Índice

1. Introducción	4
1.1. Propósito del Documento	4
1.2. Alcance del Sistema	4
1.3. Audiencia	4
2. Arquitectura del Sistema	4
2.1. Visión General	4
2.2. Diagrama de Arquitectura	5
3. Base de Datos	7
3.1. Modelo Normalizado	7
3.1.1. Tabla: clientes	7
3.1.2. Tabla: ventas	7
3.1.3. Tabla: zonas	7
3.1.4. Tabla: vendedores	7
3.1.5. Tabla: dias	8
3.1.6. Tabla: rutas	8
3.1.7. Tabla: rutas_dias (Relación N:M)	8
3.1.8. Tabla: reportes	8
3.2. Índices y Optimizaciones	9
3.3. Vista de Compatibilidad	9
4. Lógica de Negocio	11
4.1. Reglas de Validación de Bajas	11
4.2. Algoritmo de Validación	11
5. API REST	13
5.1. Endpoints de Bajas	13
5.1.1. POST /api/bajas/solicitar	13
5.1.2. GET /api/bajas/estadisticas	14
5.2. Endpoints de Planificación	14
5.2.1. GET /api/planificacion/rutas	14
5.2.2. POST /api/planificacion/migrar	14
5.2.3. POST /api/planificacion/sincronizar	15
5.2.4. DELETE /api/planificacion/limpiar	15
5.2.5. GET /api/planificacion/zonas	15
5.2.6. GET /api/planificacion/vendedores	16
5.2.7. GET /api/planificacion/dias	16
6. Sincronización con Google Sheets	17
6.1. Configuración	17
6.2. Proceso de Sincronización	17
6.3. Función de Agrupación	17

7. Seguridad	19
7.1. Autenticación de Supervisores	19
7.2. Rate Limiting	19
7.3. Validación de Archivos	19
7.4. CORS	19
7.5. Headers de Seguridad	20
8. Generación de Reportes	21
8.1. Formato Excel	21
8.2. Ubicación	21
8.3. Proceso de Generación	21
9. Deployment y Producción	23
9.1. Requisitos del Sistema	23
9.2. Variables de Entorno	23
9.3. Instalación	23
9.4. Configuración Nginx	24
10.Mantenimiento	25
10.1. Logs	25
10.2. Backup	25
10.3. Monitoreo	25
11.Troubleshooting	27
11.1. Problemas Comunes	27
11.1.1. Error: Cannot connect to MySQL	27
11.1.2. Error: Port 3001 already in use	27
11.1.3. Sincronización falla	27
11.1.4. Fotos no se cargan	27
12.Apéndices	28
12.1. Scripts Útiles	28
12.1.1. Generar Hash de Supervisor	28
12.1.2. Test de Google Sheets	28
12.1.3. Migración de Base de Datos	28
12.2. Comandos NPM	28
12.3. Estructura de Archivos	28
13.Control de Versiones	29
14.Contacto y Soporte	29

1. Introducción

1.1. Propósito del Documento

Este documento proporciona una descripción técnica completa del Sistema de Inhabilitación de Clientes de Cruzimex Ltda., incluyendo arquitectura, diseño de base de datos, lógica de negocio, APIs y procesos de sincronización.

1.2. Alcance del Sistema

El sistema permite a vendedores solicitar la inhabilitación de clientes mediante una aplicación web, validando automáticamente las solicitudes según reglas de negocio predefinidas y generando reportes para supervisión.

1.3. Audiencia

Este documento está dirigido a:

- Desarrolladores que mantendrán o extenderán el sistema
- Arquitectos de software
- Administradores de sistemas
- Equipo de TI de Cruzimex Ltda.

2. Arquitectura del Sistema

2.1. Visión General

El sistema utiliza una arquitectura cliente-servidor con las siguientes capas:

Stack Tecnológico**Frontend**

- React 18.3.1
- Vite 6.0.3
- TailwindCSS 3.4.17
- React Router DOM 7.1.1
- Axios 1.7.9
- SweetAlert2 11.15.2

Backend

- Node.js 18+
- Express 5.1.0
- MySQL2 3.13.0
- Multer 2.0.2
- BCryptJS 3.0.3
- Node-Cron 3.0.3
- ExcelJS 4.4.0

Base de Datos

- MySQL 8.0+
- Charset: utf8mb4_unicode_ci

2.2. Diagrama de Arquitectura**CAPA DE PRESENTACIÓN**

Vendedor	Supervisor	Administrador
Dashboard	Dashboard	Dashboard

React + Vite + TailwindCSS + React Router

HTTP/REST API**CAPA DE APLICACIÓN****Express.js Server**

Routes	Middleware	Services
/bajas	- Auth	- Validator
/ventas	- Upload	- ReportGen
/plani...	- Limiter	- SyncService

MySQL2 Driver

CAPA DE DATOS

MySQL Database

Clientes	Ventas	Rutas	Reportes
----------	--------	-------	----------

Zonas	Vendedores	Días	RutasDías
-------	------------	------	-----------

CAPA DE INTEGRACIÓN:

Google	Sync Service	MySQL
Sheets API	(Node-Cron)	(6AM y 7PM)

3. Base de Datos

3.1. Modelo Normalizado

El sistema utiliza un modelo de base de datos normalizado (3FN) para optimizar el almacenamiento y evitar redundancias.

3.1.1. Tabla: clientes

Campo	Tipo	Descripción
codigo	VARCHAR(50) PK	Código único del cliente
nombre	VARCHAR(255)	Nombre del cliente
ruta	VARCHAR(100)	Código de ruta asignada
zona	VARCHAR(100)	Zona geográfica
fecha_creacion	TIMESTAMP	Fecha de creación del registro

Cuadro 1: Estructura de tabla clientes

3.1.2. Tabla: ventas

Campo	Tipo	Descripción
id	INT AI PK	ID auto-incremental
codigo_cliente	VARCHAR(50)	FK a clientes.codigo
nombre_cliente	VARCHAR(255)	Nombre del cliente
fecha	DATE	Fecha de la venta
monto	DECIMAL(10,2)	Monto de la venta
producto	VARCHAR(255)	Descripción del producto
fecha_creacion	TIMESTAMP	Fecha de registro

Cuadro 2: Estructura de tabla ventas

3.1.3. Tabla: zonas

Campo	Tipo	Descripción
id	INT AI PK	ID auto-incremental
codigo	VARCHAR(50) UNIQUE	Código único de zona
nombre	VARCHAR(100)	Nombre de la zona
fecha_creacion	TIMESTAMP	Fecha de creación

Cuadro 3: Estructura de tabla zonas

3.1.4. Tabla: vendedores

Campo	Tipo	Descripción
id	INT AI PK	ID auto-incremental
nombre_completo	VARCHAR(150)	Nombre completo del vendedor
activo	BOOLEAN	Estado del vendedor

fecha_creacion	TIMESTAMP	Fecha de creación
----------------	-----------	-------------------

Cuadro 4: Estructura de tabla vendedores

3.1.5. Tabla: dias

Campo	Tipo	Descripción
id	INT AI PK	ID auto-incremental
codigo	VARCHAR(10) UNIQUE	Código del día (1-LU, 2-MA, etc.)
nombre	VARCHAR(20)	Nombre del día
orden	INT	Orden del día (1-6)

Cuadro 5: Estructura de tabla dias

3.1.6. Tabla: rutas

Campo	Tipo	Descripción
id	INT AI PK	ID auto-incremental
codigo	VARCHAR(100) UNIQUE	Código único de ruta
zona_id	INT FK	FK a zonas.id
vendedor_id	INT FK	FK a vendedores.id
fecha_creacion	TIMESTAMP	Fecha de creación
fecha_actualizacion	TIMESTAMP	Fecha de última actualización

Cuadro 6: Estructura de tabla rutas

3.1.7. Tabla: rutas_dias (Relación N:M)

Campo	Tipo	Descripción
id	INT AI PK	ID auto-incremental
ruta_id	INT FK	FK a rutas.id
dia_id	INT FK	FK a dias.id
fecha_creacion	TIMESTAMP	Fecha de creación

Cuadro 7: Estructura de tabla rutas_dias

Constraint Único: (ruta_id, dia_id) - Una ruta no puede asignarse dos veces al mismo día

3.1.8. Tabla: reportes

Campo	Tipo	Descripción
id	INT AI PK	ID auto-incremental
codigo_cliente	VARCHAR(50)	Código del cliente
nombre_cliente	VARCHAR(255)	Nombre del cliente
motivo	VARCHAR(255)	Motivo de la solicitud
zona	VARCHAR(100)	Zona del cliente

ruta	VARCHAR(100)	Ruta del cliente
vendedor	VARCHAR(150)	Vendedor asignado
resultado	VARCHAR(50)	SI, NO o DERIVADO
razon	TEXT	Razón del resultado
fotos_rutas	TEXT	Rutas de fotos (JSON)
fecha_solicitud	TIMESTAMP	Fecha de solicitud

Cuadro 8: Estructura de tabla reportes

3.2. Índices y Optimizaciones

```

1 -- Clientes
2 CREATE INDEX idx_cliente_codigo ON clientes(codigo);
3 CREATE INDEX idx_cliente_ruta ON clientes(ruta);
4
5 -- Ventas
6 CREATE INDEX idx_venta_cliente ON ventas(codigo_cliente);
7 CREATE INDEX idx_venta_fecha ON ventas(fecha);
8 CREATE INDEX idx_venta_cliente_fecha
9   ON ventas(codigo_cliente, fecha DESC);
10
11 -- Zonas
12 CREATE INDEX idx_zona_codigo ON zonas(codigo);
13
14 -- Rutas
15 CREATE INDEX idx_ruta_codigo ON rutas(codigo);
16 CREATE INDEX idx_ruta_zona ON rutas(zona_id);
17 CREATE INDEX idx_ruta_vendedor ON rutas(vendedor_id);
18
19 -- Rutas-D as
20 CREATE INDEX idx_rutadia_ruta ON rutas_dias(ruta_id);
21 CREATE INDEX idx_rutadia_dia ON rutas_dias(dia_id);
22
23 -- Reportes
24 CREATE INDEX idx_reporte_fecha ON reportes(fecha_solicitud);
25 CREATE INDEX idx_reporte_resultado ON reportes(resultado);

```

Listing 1: Índices principales

3.3. Vista de Compatibilidad

Para mantener compatibilidad con código legacy, se creó una vista que simula la tabla antigua:

```

1 CREATE OR REPLACE VIEW planificacion_rutas AS
2 SELECT
3   r.id,
4   r.codigo as ruta,
5   z.codigo as zona,
6   d.codigo as dia,
7   COALESCE(v.nombre_completo, '') as vendedor,
8   r.fecha_actualizacion as fecha_sincronizacion
9 FROM rutas r
10 INNER JOIN zonas z ON r.zona_id = z.id
11 LEFT JOIN vendedores v ON r.vendedor_id = v.id

```

```
12 INNER JOIN rutas_dias rd ON r.id = rd.ruta_id  
13 INNER JOIN dias d ON rd.dia_id = d.id  
14 ORDER BY z.codigo, d.orden, r.codigo;
```

Listing 2: Vista planificacion_rutas

4. Lógica de Negocio

4.1. Reglas de Validación de Bajas

El sistema implementa tres reglas principales para determinar si un cliente puede ser inhabilitado:

Regla 1: Sin Ventas Registradas

Condición: El cliente no tiene ninguna venta registrada en la base de datos.

Resultado: APROBADO (SI)

Razón: "No tiene ventas registradas"

Regla 2: Última Venta >90 Días

Condición: La última venta del cliente fue hace más de 90 días.

Resultado: APROBADO (SI)

Razón: "Última venta hace X días (DD/MM/YYYY)"

Regla 3: Última Venta < 90 Días (No Duplicado)

Condición: La última venta fue hace 90 días o menos Y el motivo NO incluye "duplicado"

Resultado: RECHAZADO (NO)

Razón: "Última venta hace X días (DD/MM/YYYY)"

Regla 4: Duplicado con Ventas Recientes

Condición: La última venta fue hace 90 días o menos Y el motivo incluye "duplicado"

Resultado: DERIVADO A REVISIÓN MANUAL

Razón: "Derivado a revisión manual con Inteligencia Comercial. Última venta hace X días (DD/MM/YYYY)"

4.2. Algoritmo de Validación

```
1 export const validarClienteMySQL = async (codigoCliente, motivo) => {
2   // 1. Buscar cliente
3   const cliente = await Cliente.getByCodigo(codigoCliente);
4
5   // 2. Buscar ventas del cliente
6   const ventas = await Venta.getByCliente(codigoCliente);
7
8   // 3. Obtener información de ruta, zona y vendedor
9   const rutaInfo = await PlanificacionRuta.getByRuta(
10     cliente?.ruta
11   );
12
13   // 4. CASO 1: Sin ventas registradas
14   if (ventas.length === 0) {
15     return {
```

```
16     resultado: "SI",
17     razon: "No tiene ventas registradas"
18   };
19 }
20
21 // 5. Cliente con ventas - obtener la m s reciente
22 const ultimaVenta = ventas[0]; // Ordenado DESC
23 const diasTranscurridos = calcularDias(ultimaVenta.fecha);
24
25 // 6. CASO 2: ltima venta > 90 d as
26 if (diasTranscurridos > 90) {
27   return {
28     resultado: "SI",
29     razon: ' ltima venta hace ${diasTranscurridos} d as '
30   };
31 }
32
33 // 7. CASO 3 y 4: ltima venta <= 90 d as
34 const esDuplicado = motivo.toLowerCase()
35   .includes("duplicado");
36
37 if (esDuplicado) {
38   return {
39     resultado: "DERIVADO A REVISI N MANUAL",
40     razon: "Derivado a Inteligencia Comercial"
41   };
42 }
43
44 return {
45   resultado: "NO",
46   razon: ' ltima venta hace ${diasTranscurridos} d as '
47 };
48 };
```

Listing 3: Algoritmo de validación (validatorMySQL.js)

5. API REST

5.1. Endpoints de Bajas

5.1.1. POST /api/bajas/solicitar

Solicita la inhabilitación de un cliente.

Headers:

- Content-Type: multipart/form-data

Body (form-data):

```
1 {
2   "codigoCliente": "12345",
3   "motivo": "Cliente duplicado",
4   "fotos": [File, File, ...] // 1-5 archivos
5 }
```

Respuesta Exitosa (200):

```
1 {
2   "puedeInhabilitar": true,
3   "mensaje": "Cliente puede ser inhabilitado",
4   "codigo": "12345",
5   "nombreCliente": "JUAN PEREZ",
6   "motivo": "Cliente duplicado",
7   "razon": " ltima  venta hace 95 d as (15/08/2024)",
8   "detalles": {
9     "zona": "SC DTS 1",
10    "ruta": "SC-RUTA 11",
11    "vendedor": "PEDRO MARTINEZ"
12  }
13 }
```

Respuesta Rechazada (200):

```
1 {
2   "puedeInhabilitar": false,
3   "mensaje": "Cliente NO puede ser inhabilitado",
4   "codigo": "12345",
5   "nombreCliente": "MARIA LOPEZ",
6   "motivo": "Cliente inactivo",
7   "razon": " ltima  venta hace 30 d as (15/10/2024)"
8 }
```

Respuesta Derivada (200):

```
1 {
2   "puedeInhabilitar": false,
3   "requiereRevisionManual": true,
4   "mensaje": "Solicitud derivada a revisi n manual",
5   "codigo": "12345",
6   "nombreCliente": "CARLOS GOMEZ",
7   "motivo": "Cliente duplicado",
8   "razon": "Derivado a revisi n manual...",
9   "instrucciones": [
10    "1. La solicitud ha sido registrada",
11    "2. Ser revisado por Inteligencia Comercial",
12    "3. Respuesta en 2-4 horas",
13    "4. Consultar con supervisor"
14  ]
15 }
```

```
14 ]
15 }
```

5.1.2. GET /api/bajas/estadisticas

Obtiene estadísticas del día actual.

Autenticación: No requiere. Acceso público para consulta.

Uso: Esta API se muestra automáticamente en el dashboard administrativo al descargar reportes.

Respuesta (200):

```
1 {
2   "fecha": "2024-11-25",
3   "total": 150,
4   "aprobadas": 45,
5   "rechazadas": 80,
6   "derivadas": 25,
7   "porcentajeAprobacion": 30.0
8 }
```

5.2. Endpoints de Planificación

5.2.1. GET /api/planificacion/rutas

Obtiene todas las rutas de planificación.

Respuesta (200):

```
1 {
2   "success": true,
3   "total": 375,
4   "rutas": [
5     {
6       "id": 1,
7       "codigo": "SC-RUTA 11",
8       "zona_codigo": "SC DTS 1",
9       "zona_nombre": "SC DTS 1",
10      "vendedor_nombre": "ALEJANDRA TANIA GUANACO PAYE",
11      "dias_codigos": "3-MI, 5-VI",
12      "dias_nombres": "Mi rcoles , Viernes"
13     }
14   ]
15 }
```

5.2.2. POST /api/planificacion/migrar

Ejecuta la migración inicial de datos desde Google Sheets.

Autenticación: **REQUERIDA**. Debe enviarse `codigoSupervisor` en el body.

Seguridad: Esta ruta está protegida con el middleware `authenticateSupervisor`. Solo accesible desde la interfaz administrativa, NO mediante URL directa.

Body:

```
1 {
2   "codigoSupervisor": "CODIGO_HASHEADO"
3 }
```

Respuesta (200):

```
1 {
2   "success": true,
3   "message": "Migraci n completada",
4   "insertados": 375,
5   "errores": 0
6 }
```

5.2.3. POST /api/planificacion/sincronizar

Sincroniza datos actuales desde Google Sheets.

Autenticación: **REQUERIDA**. Debe enviarse `codigoSupervisor` en el body.

Seguridad: Esta ruta está protegida con el middleware `authenticateSupervisor`. Solo accesible desde la interfaz administrativa, NO mediante URL directa.

Body:

```
1 {
2   "codigoSupervisor": "CODIGO_HASHEADO"
3 }
```

Respuesta (200):

```
1 {
2   "success": true,
3   "message": "Sincronizaci n completada",
4   "insertados": 5,
5   "actualizados": 12,
6   "sinCambios": 358,
7   "errores": 0
8 }
```

5.2.4. DELETE /api/planificacion/limpiar

Limpia todas las tablas de planificación. **¡OPERACIÓN DESTRUCTIVA!**

Autenticación: **REQUERIDA**. Debe enviarse `codigoSupervisor` en el body.

Seguridad: Esta ruta está protegida con el middleware `authenticateSupervisor`. Solo accesible desde la interfaz administrativa, NO mediante URL directa.

Body:

```
1 {
2   "codigoSupervisor": "CODIGO_HASHEADO"
3 }
```

Respuesta (200):

```
1 {
2   "success": true,
3   "message": "Tablas limpiadas exitosamente"
4 }
```

5.2.5. GET /api/planificacion/zonas

Obtiene todas las zonas.

Autenticación: No requiere. Datos públicos de consulta.

5.2.6. GET /api/planificacion/vendedores

Obtiene todos los vendedores.

Autenticación: No requiere. Datos públicos de consulta.

5.2.7. GET /api/planificacion/dias

Obtiene todos los días de la semana.

Autenticación: No requiere. Datos públicos de consulta.

6. Sincronización con Google Sheets

6.1. Configuración

El sistema sincroniza datos de planificación desde Google Sheets mediante:

- **URL:** Configurada en .env como GOOGLE_SHEET_URL
- **Formato:** CSV exportado desde Google Sheets
- **Frecuencia:** Automática a las 6:00 AM y 7:00 PM (hora de Bolivia)
- **Alternativa:** Sincronización manual desde interfaz web

6.2. Proceso de Sincronización

1. **Lectura:** El servicio descarga el CSV desde Google Sheets
2. **Parseo:** Convierte el CSV a objetos JavaScript
3. **Normalización:** Limpia y valida los datos
4. **Agrupación:** Elimina duplicados agrupando por código de ruta
5. **Inserción/Actualización:**
 - Inserta zonas únicas
 - Inserta vendedores únicos
 - Inserta rutas con relaciones a zonas y vendedores
 - Crea relaciones rutas-días (N:M)
6. **Registro:** Guarda log de sincronización

6.3. Función de Agrupación

```
1 function agruparPorRuta(registros) {
2   const rutasMap = new Map();
3
4   for (const registro of registros) {
5     const normalizado = normalizarRegistro(registro);
6
7     if (!rutasMap.has(normalizado.ruta)) {
8       // Primera vez que vemos esta ruta
9       rutasMap.set(normalizado.ruta, {
10         codigo: normalizado.ruta,
11         zona: normalizado.zona,
12         vendedor: normalizado.vendedor,
13         dias: new Set([normalizado.dia])
14       });
15     } else {
16       // La ruta ya existe, agregar el d a
17       const rutaExistente = rutasMap.get(normalizado.ruta);
18       rutaExistente.dias.add(normalizado.dia);
19     }
20   }
21 }
```

```
20 }  
21  
22 return rutasMap;  
23 }
```

Listing 4: Eliminación de duplicados

Ejemplo:

Entrada (392 registros CSV):

```
SC-RUTA 11, SC DTS 1, 3-MI, ALEJANDRA PAYE  
SC-RUTA 11, SC DTS 1, 5-VI, ALEJANDRA PAYE  <- DUPLICADO  
SC-RUTA 143, SC DTS 1, 3-MI, CRISTIAN VACA
```

Salida (2 rutas únicas):

Ruta 1: SC-RUTA 11
- Zona: SC DTS 1
- Vendedor: ALEJANDRA PAYE
- Días: [3-MI, 5-VI]

Ruta 2: SC-RUTA 143
- Zona: SC DTS 1
- Vendedor: CRISTIAN VACA
- Días: [3-MI]

7. Seguridad

7.1. Autenticación de Supervisores

- **Método:** Códigos hasheados con BCrypt
- **Almacenamiento:** Base de datos (tabla `supervisores`). Solo el código principal se guarda en variable de entorno `ADMIN_CODE`
- **Validación:** Middleware `validateSupervisor`
- **Tiempo de sesión:** 24 horas (renovable)
- **Gestión:** Los supervisores son creados por el administrador desde la interfaz web

7.2. Rate Limiting

```
1 // General: 100 requests por 15 minutos
2 const generalLimiter = rateLimit({
3   windowMs: 15 * 60 * 1000,
4   max: 100
5 });
6
7 // Bajas: 10 solicitudes por 15 minutos
8 const bajasLimiter = rateLimit({
9   windowMs: 15 * 60 * 1000,
10  max: 10
11 });
```

Listing 5: Configuración de rate limit

7.3. Validación de Archivos

- **Formatos permitidos:** JPG, JPEG, PNG, WEBP
- **Tamaño máximo por archivo:** 5 MB
- **Cantidad máxima:** 5 archivos por solicitud
- **Sanitización:** Nombres de archivo sanitizados

7.4. CORS

```
1 const corsOptions = {
2   origin: process.env.FRONTEND_URL || 'http://localhost:5173',
3   methods: ['GET', 'POST', 'PUT', 'DELETE'],
4   credentials: true
5 };
```

7.5. Headers de Seguridad

- Helmet.js para headers HTTP seguros
- Content-Security-Policy
- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY

8. Generación de Reportes

8.1. Formato Excel

Los reportes se generan en formato Excel (.xlsx) con las siguientes columnas:

Columna	Descripción
Fecha	Fecha y hora de la solicitud
Código Cliente	Código del cliente solicitado
Nombre Cliente	Nombre completo del cliente
Motivo	Motivo de inhabilitación ingresado
Zona	Zona geográfica del cliente
Ruta	Ruta asignada al cliente
Vendedor Asignado (Planif.)	Vendedor según planificación
Vendedor Día Solicitud	Vendedor el día de la solicitud
Resultado	SI, NO o DERIVADO
Razón	Razón detallada del resultado

Cuadro 9: Columnas del reporte Excel

8.2. Ubicación

- **Archivo:** backend/reportes/reporte_supervisores.xlsx
- **Actualización:** Tiempo real con cada solicitud
- **Formato:** Excel 2007+ (.xlsx)

8.3. Proceso de Generación

```

1 export async function agregarSolicitudAlReporte(resultado) {
2   const filePath = path.join(__dirname,
3     '../reportes/reporte_supervisores.xlsx');
4
5   let workbook;
6   if (fs.existsSync(filePath)) {
7     workbook = new ExcelJS.Workbook();
8     await workbook.xlsx.readFile(filePath);
9   } else {
10    workbook = new ExcelJS.Workbook();
11    const worksheet = workbook.addWorksheet('Solicitudes');
12    worksheet.columns = [
13      { header: 'Fecha', key: 'fecha', width: 20 },
14      { header: 'Código Cliente', key: 'codigo', width: 15 },
15      // ... m s columnas
16    ];
17  }
18
19  const worksheet = workbook.getWorksheet('Solicitudes');
20  worksheet.addRow({
21    fecha: new Date().toLocaleString('es-B0'),
22    codigo: resultado.codigoCliente,
23    // ... m s datos

```

```
24     });  
25  
26     await workbook.xlsx.writeFile(filePath);  
27 }
```

Listing 6: Generación de reporte

9. Deployment y Producción

9.1. Requisitos del Sistema

Hardware Mínimo:

- CPU: 2 cores
- RAM: 4 GB
- Disco: 20 GB (SSD recomendado)

Software:

- Node.js 18+ o 20+
- MySQL 8.0+
- Sistema operativo: Linux (Ubuntu 20.04+ recomendado) o Windows Server

9.2. Variables de Entorno

```
1 # Servidor
2 PORT=3001
3 NODE_ENV=production
4
5 # Base de datos
6 DB_HOST=localhost
7 DB_PORT=3306
8 DB_USER=cruzimex_user
9 DB_PASSWORD=<PASSWORD_SEGURO>
10 DB_NAME=sistema_bajas
11
12 # Frontend
13 FRONTEND_URL=https://bajas.cruzimex.com
14
15 # Google Sheets
16 GOOGLE_SHEET_URL=https://docs.google.com/spreadsheets/...
17
18 # Seguridad (Código del administrador principal)
19 ADMIN_CODE=<HASH_BCRYPT_DEL_CODIGO_ADMIN>
```

Listing 7: .env producción

9.3. Instalación

```
1 # 1. Clonar repositorio
2 git clone <REPO_URL>
3 cd SistemaDeBajas
4
5 # 2. Instalar dependencias backend
6 cd backend
7 npm install --production
8
9 # 3. Configurar .env
10 cp .env.example .env
```

```
11 nano .env # Editar variables
12
13 # 4. Ejecutar migraciones
14 npm run init-db
15 npm run migrate
16
17 # 5. Construir frontend
18 cd ../frontend
19 npm install
20 npm run build
21
22 # 6. Configurar Nginx
23 sudo cp nginx.conf /etc/nginx/sites-available/bajas
24 sudo ln -s /etc/nginx/sites-available/bajas \
25     /etc/nginx/sites-enabled/
26 sudo nginx -t
27 sudo systemctl reload nginx
28
29 # 7. Configurar PM2
30 cd ../backend
31 pm2 start index.js --name "bajas-backend"
32 pm2 save
33 pm2 startup
```

9.4. Configuración Nginx

```
1 server {
2     listen 80;
3     server_name bajas.cruzimex.com;
4
5     # Frontend (React build)
6     location / {
7         root /var/www/bajas/frontend/dist;
8         try_files $uri /index.html;
9     }
10
11     # Backend API
12     location /api/ {
13         proxy_pass http://localhost:3001;
14         proxy_http_version 1.1;
15         proxy_set_header Upgrade $http_upgrade;
16         proxy_set_header Connection 'upgrade';
17         proxy_set_header Host $host;
18         proxy_cache_bypass $http_upgrade;
19         proxy_set_header X-Real-IP $remote_addr;
20         proxy_set_header X-Forwarded-For
21             $proxy_add_x_forwarded_for;
22     }
23
24     # Uploads (fotos)
25     location /uploads/ {
26         alias /var/www/bajas/backend/uploads/;
27     }
28 }
```

Listing 8: /etc/nginx/sites-available/bajas

10. Mantenimiento

10.1. Logs

Ubicaciones:

- Backend logs: backend/logs/
- PM2 logs: ~/.pm2/logs/
- Nginx logs: /var/log/nginx/

Rotación:

```
1 # PM2 log rotation
2 pm2 install pm2-logrotate
3 pm2 set pm2-logrotate:max_size 10M
4 pm2 set pm2-logrotate:retain 30
```

10.2. Backup

Base de Datos (diario):

```
1 #!/bin/bash
2 # backup-mysql.sh
3 DATE=$(date +%Y%m%d_%H%M%S)
4 mysqldump -u root -p sistema_bajas \
5 > /backups/sistema_bajas_${DATE}.sql
6 gzip /backups/sistema_bajas_${DATE}.sql
7
8 # Eliminar backups > 30 d as
9 find /backups/ -name "*.sql.gz" -mtime +30 -delete
```

Archivos (semanal):

```
1 #!/bin/bash
2 # backup-files.sh
3 DATE=$(date +%Y%m%d)
4 tar -czf /backups/uploads_${DATE}.tar.gz \
5 /var/www/bajas/backend/uploads/
6 tar -czf /backups/reportes_${DATE}.tar.gz \
7 /var/www/bajas/backend/reportes/
```

10.3. Monitoreo

Métricas a monitorear:

- Uso de CPU y RAM
- Espacio en disco
- Conexiones MySQL
- Tiempo de respuesta API
- Errores HTTP 5xx
- Tasa de sincronización exitosa

Alertas:

- CPU $\geq 80\%$ por 5 minutos
- RAM $\geq 90\%$
- Disco $\leq 10\%$ libre
- Tasa de error $\geq 5\%$

11. Troubleshooting

11.1. Problemas Comunes

11.1.1. Error: Cannot connect to MySQL

Causa: Credenciales incorrectas o MySQL no está corriendo.

Solución:

```
1 # Verificar estado
2 sudo systemctl status mysql
3
4 # Verificar conexión
5 mysql -u root -p -e "SELECT 1;"
6
7 # Verificar .env
8 cat backend/.env | grep DB_
```

11.1.2. Error: Port 3001 already in use

Causa: El puerto ya está ocupado.

Solución:

```
1 # Encontrar proceso
2 lsof -i :3001
3
4 # Matar proceso
5 kill -9 <PID>
6
7 # O cambiar puerto en .env
8 PORT=3002
```

11.1.3. Sincronización falla

Causa: Google Sheet no es público o URL incorrecta.

Solución:

1. Verificar que el Google Sheet sea público
2. Ejecutar test: node scripts/testGoogleSheets.js
3. Revisar logs del scheduler

11.1.4. Fotos no se cargan

Causa: Permisos incorrectos en carpeta uploads.

Solución:

```
1 # Dar permisos
2 chmod 755 backend/uploads
3 chown www-data:www-data backend/uploads
4
5 # Verificar Nginx
6 ls -la /var/www/bajas/backend/uploads/
```

12. Apéndices

12.1. Scripts Útiles

12.1.1. Generar Hash de Supervisor

```
1 cd backend
2 node scripts/generateHash.js
```

12.1.2. Test de Google Sheets

```
1 cd backend
2 node scripts/testGoogleSheets.js
```

12.1.3. Migración de Base de Datos

```
1 cd backend
2 npm run migrate
```

12.2. Comandos NPM

```
1 # Backend
2 npm start          # Iniciar servidor producci n
3 npm run dev        # Iniciar con nodemon
4 npm run migrate    # Ejecutar migraci n DB
5 npm run generate-hash # Generar hash supervisor
6
7 # Frontend
8 npm run dev        # Servidor desarrollo
9 npm run build      # Construir para producci n
10 npm run preview    # Preview de build
```

12.3. Estructura de Archivos

```
SistemaDeBajas/
  backend/
    config/
      database.js
      mysql.js
    middleware/
      auth.js
      errorHandler.js
      rateLimiter.js
      uploadHandler.js
    models/
      Cliente.js
      Dia.js
      Reporte.js
      Ruta.js
      SyncLog.js
```

```
Vendedor.js
Venta.js
Zona.js
routes/
  bajas.js
  clientes.js
  index.js
  planificacion.js
  supervisores.js
  ventas.js
services/
  googleSheetsReader.js
  planificacionSyncService.js
  reportGenerator.js
  scheduler.js
  supervisorService.js
  validatorMySQL.js
migrations/
  normalize_planificacion.sql
  README.md
scripts/
  generateHash.js
  initDatabase.js
  runMigration.js
  testGoogleSheets.js
index.js
package.json
.env
frontend/
  src/
    components/
    utils/
    App.jsx
    main.jsx
  public/
    index.html
    package.json
    vite.config.js
```

13. Control de Versiones

14. Contacto y Soporte

Desarrollador: Ludwing Julian Herrera Justiniano

Empresa: Cruzimex Ltda.

Repositorio: <https://github.com/Ludwingh28/SistemaDeBajas>

Versión	Fecha	Autor	Cambios
1.0	10-11-2025	L. Herrera	Versión inicial
2.0	25-11-2025	L. Herrera	Normalización BD, Google Sheets sync, documentación completa

Cuadro 10: Historial de versiones